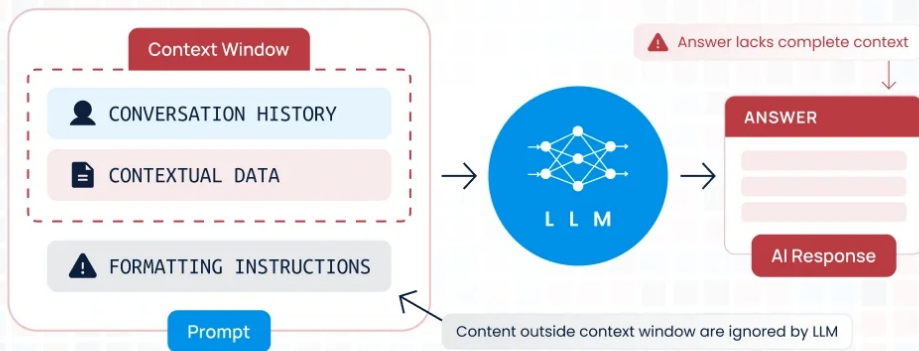


Context Window Rot: Why More Context Hurts

Context Window Limitation



What Is Context Window Rot?

Stanford's "Lost in the Middle" (2023) study found that LLMs reliably retrieve information from the beginning and end of context windows but consistently miss information placed in the middle, regardless of model size.

Retrieval Failure

The model receives 15 context chunks but uses information only from chunks 1-3 and 13-15. Chunks 4-12 in the middle are effectively ignored. The retrieved information was there but not used in the answer.

Context Dilution

Including 20 low-relevance chunks alongside 3 high-relevance chunks degrades answer quality even when the high-relevance chunks are at the start. Too much noise overwhelms the signal.

Position Bias

For the same query and same chunks, placing the key information at position 1 vs position 10 changes answer accuracy by 20-28 percentage points in controlled experiments. Position is a first-class concern.



Lost-in-Middle Evidence by Position

Context Position	Answer Accuracy	Relative to Best
Position 1 (start)	82%	Baseline (best)
Position 2-3	76%	-7%
Position 4-7	62%	-24%
Position 8-12 (middle)	54%	-34% (worst)
Position 13-16	63%	-23%
Position 20 (end)	79%	-4%

Why This Happens

Transformer attention is biased toward recent tokens (recency bias) and the first tokens set context (primacy bias). Tokens in the middle of a long sequence compete with many surrounding tokens and receive proportionally less attention during generation.



Source: Stanford 2023 "Lost in the Middle." Applies to GPT-4, Claude, PaLM, and all major transformer-based LLMs regardless of stated context window size.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

 Repost

analyze_context Relevance and Position

```
import tiktoken, numpy as np

enc = tiktoken.get_encoding("cl100k_base")

def analyze_context(
    chunks: list[str],
    query_emb: list[float],
    embedder
) → list[dict]:
    n = len(chunks)
    results = []
    for i, chunk in enumerate(chunks):
        tokens = len(enc.encode(chunk))
        chunk_emb = embedder.embed(chunk)
        relevance = float(np.dot(query_emb, chunk_emb) /
                           (np.linalg.norm(query_emb) * np.linalg.norm(chunk_emb)))
        position = i / n
        band = ("start" if position < 0.25 else
               "end" if position > 0.75 else
               "middle")
        results.append({"chunk_id": i, "tokens": tokens,
                       "relevance": relevance, "band": band,
                       "low_rel_middle": band == "middle" and relevance < 0

    return results
```



Flag low_rel_middle chunks before assembly. These are the worst possible chunks: low relevance AND worst position. Remove them before passing context to the LLM.



position_aware_context_assembly **Best-First-and-Last**

```
def position_aware_context_assembly(
    chunks_with_scores: list[tuple[str, float]],
    token_budget: int = 8000
) → str:
    # Sort by relevance descending
    sorted_chunks = sorted(chunks_with_scores,
                           key=lambda x: x[1], reverse=True)
    # Interleave: best at start, second-best at end, etc.
    n = len(sorted_chunks)
    result = [None] * n
    left, right = 0, n - 1

    for i, (chunk, _) in enumerate(sorted_chunks):
        if i % 2 == 0:
            result[left] = chunk; left += 1
        else:
            result[right] = chunk; right -= 1

    # Enforce token budget from the assembled context
    assembled, total = [], 0
    for chunk in result:
        t = len(enc.encode(chunk))
```

✓ Quality Impact

In controlled testing, best-first-and-last assembly improves faithfulness by 8-15% over naive top-N assembly for queries requiring information from multiple chunks. Especially effective for 10+ chunk contexts.



extract_relevant_span Sentence-Level Filtering

Long chunks often contain 40-60% irrelevant filler text. Sentence-level filtering keeps only the sentences that are semantically close to the query, reducing chunk token count dramatically without losing key information.

```

import re, numpy as np

def extract_relevant_span(
    chunk: str,
    query_emb: list[float],
    embedder,
    threshold: float = 0.75
) → str:
    # Split into sentences (simple regex; use nltk for production)
    sentences = re.split(r'(?<=[.!?])\s+', chunk.strip())
    kept = []
    for sent in sentences:
        if len(sent.split()) < 4: continue # skip too-short sentences
        s_emb = embedder.embed(sent)
        sim = np.dot(query_emb, s_emb) / (
            np.linalg.norm(query_emb) * np.linalg.norm(s_emb))
        if sim ≥ threshold:
            kept.append(sent)
    # Return filtered chunk; typically 40-60% smaller
    return " ".join(kept) if kept else sentences[0] # keep first as fallback

```



build_instruction_anchored_prompt Primacy and Recency

For long contexts, repeat the core instruction at both the beginning and end of the prompt. This exploits primacy (start matters) and recency (end matters) to ensure the instruction is never forgotten.

```
def build_instruction_anchored_prompt(
    instruction: str,
    context: str,
    question: str
) → str:
    # Instruction at start (primacy effect)
    prompt = f"INSTRUCTION: {instruction}\n\n"

    # Context assembled with position-aware ordering
    prompt += f"CONTEXT:\n{context}\n\n"

    # Instruction repeated at end (recency effect)
    prompt += (
        f"Remember: {instruction}\n"
        "Answer ONLY based on the context above.\n"
        "If the answer is not in the context, say: "
        "'I cannot find this information in the provided context.'\n\n"
        f"QUESTION: {question}\nANSWER:"
    )
    return prompt
```



This adds under 30 tokens to the prompt but can improve instruction following accuracy by 10-20% on long-context queries where the instruction gets buried between the start and question.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

Repost

map_reduce_qa Async for Very Long Documents

```
import asyncio

def chunk_document(text: str, max_tokens: int = 4000) → list[str]:
    words = text.split()
    # Rough approximation: 1 token ~= 0.75 words
    chunk_size = int(max_tokens * 0.75)
    return [" ".join(words[i:i+chunk_size])
            for i in range(0, len(words), chunk_size)]

async def map_reduce_qa(
    document: str,
    query: str,
    llm_client
) → str:
    chunks = chunk_document(document)
    sem = asyncio.Semaphore(10)

    async def answer_chunk(chunk: str) → str:
        async with sem:
            return await llm_client.complete(
                f"Answer based ONLY on this passage:\n{chunk}\n\nQ: {query}"

    # Map: answer each chunk independently
    partial_answers = await asyncio.gather(*[answer_chunk(c) for c in chunks])

    # Reduce: synthesize partial answers
    joined = "\n".join(f"[Part {i+1}]: {a}" for i, a in enumerate(partial_answers))
    return await llm_client.complete(f"Synthesize:\n{joined}\n\nFinal answer")
```



measure_context_length_degradation Find Your Quality Cliff

```

async def measure_context_length_degradation(
    test_queries: list[dict], # {query, ground_truth}
    context_lengths: list[int] = [2000, 4000, 8000, 16000, 32000],
    llm_client = None,
    embedder = None
) → dict[int, float]:
    results = {}
    for ctx_len in context_lengths:
        faithfulness_scores = []
        for item in test_queries:
            context = truncate_to_tokens(item["context"], ctx_len)
            answer = await llm_client.complete(
                f"{context}\n\nQ: {item['query']}"
            )
            score = compute_faithfulness(answer, context)
            faithfulness_scores.append(score)

        results[ctx_len] = round(float(np.mean(faithfulness_scores)), 4)
        log(f"ctx={ctx_len:6d} tokens: faithfulness={results[ctx_len]:.3f}")
    return results
# Find the token length where faithfulness starts dropping more than 5%

```



Run this once per model and per domain. Most models show a quality cliff between 8k-16k tokens despite supporting 128k. The cliff is the ceiling for your production context length.



AI Coach John

Follow



Repost



Context Assembly Strategy Comparison

Strategy	Best For	Token Efficiency	Implementation
Best-first-and-last	General RAG	High	position_aware_context_assembly()
Sentence filtering	Dense documents	Very high	extract_relevant_span()
Map-reduce QA	Very long docs	Medium	map_reduce_qa() async
Instruction anchoring	Complex instructions	No overhead	build_instruction_anchored_prompt()
Summary-then-detail	Hierarchical docs	High	Summarize sections, append originals

💡 Combine Techniques

For best results: apply sentence filtering first to reduce chunk sizes, then position-aware assembly for ordering, then instruction anchoring on the final prompt. All three together can improve faithfulness by 20-30% on long-context queries.



Context Length Budget **Practical Guidelines**

Context Length	Recommended Use	Expected Quality
2-4k tokens	High-precision RAG, factual QA	Best quality
4-8k tokens	Standard RAG, document analysis	Very good
8-16k tokens	Multi-document synthesis	Good (monitor)
16-32k tokens	Full document QA (with caution)	Declining
32k+ tokens	Use map-reduce instead	Do not use directly

Do Not Assume

Every model and domain has a different quality cliff. The table above is a starting baseline. Run `measure_context_length_degradation()` on your specific model, domain, and query types before finalizing your context budget policy.



AI Coach John

Follow



 Repost

Context Window Rot Mistakes

- ✖ **Naive top-N chunk concatenation.** Concatenating the top 10 chunks by relevance in order of retrieval score ignores position effects. The most relevant chunk should always be at position 0, not wherever it happened to rank.
- ✖ **Including low-relevance chunks to fill the context window.** "More context is free" is false. Low-relevance chunks dilute the signal and compete for attention with high-relevance chunks even at good positions.
- ✖ **Assuming 128k context means 128k quality.** The context window size and the context quality curve are two completely different things. A 128k-token model degrades in quality well before 128k tokens in most real tasks.
- ✖ **Not repeating instructions on long prompts.** Instructions at the top of a 20k-token prompt are effectively forgotten by the time the model generates the answer. Anchoring at both ends is a free quality improvement.
- ✖ **Chunk splitting at fixed token count.** Splitting at exactly 512 tokens regularly cuts sentences mid-thought. Use sentence-boundary-aware splitting to preserve semantic coherence within chunks.
- ✖ **Never measuring faithfulness vs context length.** Without an empirical quality curve, you do not know where the cliff is. Teams routinely use 32k-token contexts where 8k would give 20% better answers.



Context Window Rot **Full Reference**

Technique	Code	When to Use
Position-aware assembly	<code>position_aware_context_assembly()</code>	All RAG pipelines, always
Sentence-level filtering	<code>extract_relevant_span()</code>	Dense docs with padding content
Instruction anchoring	<code>build_instruction_anchored_prompt()</code>	Complex instructions over 8k tokens
Map-reduce QA	<code>map_reduce_qa() async</code>	Documents over 32k tokens
Context analysis	<code>analyze_context()</code>	Debug why answers miss middle info
Degradation measurement	<code>measure_context_length_degradation()</code>	Before finalizing context budget
Token counting	<code>tiktoken.get_encoding().encode()</code>	Enforce input budget on every call

🛠️ **Context quality beats context quantity. Use the minimum tokens that preserve the maximum information.**



🚩 KEY TAKEAWAY

Context Quality Beats Context Quantity

The 128k-token context window is a ceiling, not a target. Most production RAG pipelines perform best at 4–12k tokens of carefully assembled, position-optimized, sentence-filtered context. Less is more, if done right.

🔑 Start Today: 3 Actions

- Replace your top-N chunk concatenation with `position_aware_context_assembly()`. Ship this week.
- Run `measure_context_length_degradation()` on your pipeline. Find your actual quality cliff, not the assumed one.
- For any document over 16k tokens, switch from one-shot to `map_reduce_qa()` `async`. Consistent quality gains with no extra infrastructure.

Follow AI Coach John for more production AI engineering content.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

↻ Repost